Natural Language Processing

UniVR - Dipartimento di Informatica

Tommaso Toffali

1° Semestre 2026/2027

Indice

1 Introduction **2**
1.1 How did language emerge? . . . 2
1.2 What is language? . . . 2
1.3 Who does use language? . . . 2
1.4 Level of linguistic representation . . . 3
1.5 Typology based on writing system . . . 3
1.6 IPA . . . 3
1.7 Ambiguity . . . 4

I Language Modeling **4**

2 Introduction to N-grams **4**
2.0.1 Probabilistic Language Models . . . 4
2.0.2 Why Probabilistic Language Models? . . . 4
2.0.3 Machine Translation . . . 4
2.0.4 Spell Correction . . . 5
2.0.5 Speech Recognition . . . 5
2.0.6 Other Applications . . . 5
2.1 How to Compute P(W) . . . 5
2.1.1 Naive approach: Counting . . . 5
2.1.2 Chain Rule of Probability . . . 6
2.1.3 How to Estimate These Probabilities . . . 6
2.2 Neural Language Models . . . 7
2.2.1 Limitations of N-gram Models . . . 7
2.2.2 From N-grams to Neural Networks . . . 7
2.2.3 Feedforward Neural Language Models . . . 7
2.2.4 Recurrent Neural Language Models . . . 8
2.2.5 Long Short-Term Memory (LSTM) . . . 8

1 Introduction

Language is a structured system of conventional signs used by agents to represent, express and communicate meanings.

1.1 How did language emerge?

Two question must be distinguished:

- **fThe evolutionary origin of the human capacity for language:** how biological, cognitive and social evolution made language possibile.
- **The historical development of particular language:** how language changes, differentiate, interact and give rise to language families

The origin of language itself is not directly documented. Spoken and signed languages existed long before writing, so their early history must be reconstructured indirectly using evidence from:

- human biology and cognition
- archaeology
- comparative linguistic
- animal comunication

1.2 What is language?

Language is a structured system of conventional signs used by agents to represent, express and communicate meanings.

- Spoken sound
- Signed gesture
- Written symbols
- Tactile signs, such as Braille
- Haptic signs, conveyed through touch or movement

1.3 Who does use language?

- **Human agents**
- **Non-human animal agents**
  - some non-human animals use structured communication system, although their classification as languages is disputed
- **Artificial agents**
  - artificial agents process and generate linguistic expressions, while the nature of their understanding remains an open theoretical question.

1.4 Level of linguistic representation

Level	Principal units	Central question
Signal	Sounds, gestures, written or tactile signs	How is language physically conveyed?
Phonological or graphemic	Phonemes, syllables, graphemes	How are elementary signs organized and distinguished?
Morphological	Morphemes and word forms	How are meaningful components combined into words?
Lexical	Words, lemmas and senses	What words exist, and what can they mean?
Syntactic	Phrases, clauses and sentences	How are words combined into structured expressions?
Semantic	Concepts, entities, predicates, arguments and propositions	What does the expression mean?
Discourse	Sentences, utterances and discourse relations	How does an expression connect with the surrounding text or conversation?
Pragmatic	Speakers, hearers, intentions and context	What does an agent want to obtain by using the expression in this situation?

1.5 Typology based on writing system

Languages should be distinguished based on their features. We may start by their writing system to step onto four further basic features:

- Morphology
- Syntax
- Semantics
- Pragmatics

1.6 IPA

The international phonetic alphabet (IPA) is a standardized system for representing the sounds of spoken language. Ideally, each symbol identifies a particular speech sound independently of the ordinary spelling conventions of a laguage.

We can present three languages as a progression in the relationship between **written form and pronounciation**. This is not a progression from a *simple language* to a *complex language*, it concerns the complexity of their orthographic systems and form-sound correspondences.

1.7 Ambiguity

Ambiguity occurs when the same linguistic expression permits two or more distinct analyses or interpretations. **RECUPERARE FINO A SLIDE 66 NLP.pdf**

Parte I
Language Modeling

2 Introduction to N-grams

2.0.1 Probabilistic Language Models

A **Language Model** (LM) is a model that assigns probabilities to sequences of words.

Goal: Compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5, \ldots, w_n)$$

Related task: Probability of an upcoming word:

$$P(w_5 \mid w_1, w_2, w_3, w_4)$$

A model that computes either of these is called a **language model**.

Terminology note: Better term would be "the grammar" but **language model** or **LM** is standard.

2.0.2 Why Probabilistic Language Models?

Language models are fundamental for many NLP applications.

2.0.3 Machine Translation

A language model helps choose the most natural translation.

Esempio 2.1 (Natural translation)

- P(high winds tonite) > P(large winds tonite)

"High winds" is a more common collocation than "large winds"

2.0.4 Spell Correction

Esempio 2.2 (Spell Correction)

"The office is about fifteen minutes from my house"

$$P(\text{about fifteen } \textbf{minutes} \text{ from }) > P(\text{about fifteen } \textbf{minutes} \text{ from})$$

Both "minutes" and "minutes" are valid words, but the context makes "minutes" much more likely.

2.0.5 Speech Recognition

Many word sequences sound similar when spoken.

Esempio 2.3 (Speech Recognition)

$$P(I\text{saw a van }) \gg P(\text{eyes awe of an})$$

The labguage model helps disambiguate between phonetically similar sequences

2.0.6 Other Applications

- **Summarization**
- **Question-answering**
- **Dialogue systems**
- **Text generation**
- And many more!

2.1 How to Compute P(W)

How do we compute the probability of a word sequence?

Esempio 2.4 (How to compute)

How to compute this joint probability:

$$P(\text{its, water, is, so, transparent, that})$$

2.1.1 Naive approach: Counting

We could how many times this exact sequence appears in a corpus:

$$P(W) = \frac{\text{Count}(w_1, w_2, \ldots, w_n)}{\text{Total number of word sequences of lenght } n}$$

Problems:

- **Data sparsity**: Most sequences never appear in any corpus

- **Combinatorial explosion**: Number of possible sequences grows exponentially
- **No generalization**: Can't handle novel sequences

2.1.2 Chain Rule of Probability

Reminder: Recall the definition of conditional probabilities:

$$p(B \mid A) = \frac{P(A, B)}{P(A)}$$

Rewriting this: $P(A \mid B) = P(A) \cdot P(B \mid A)$

Generalizing to more variables:

$$P(A, B, C, D) = P(A) \cdot P(B \mid A) \cdot P(C \mid A, B) \cdot P(D \mid A, B, C)$$

The chain rule in general:

$$P(x_1, x_2, x_3, \ldots, x_n) = P(x_1) \cdot P(x_2 \mid x_1) \cdot P(x_3 \mid x_1, x_2) \ldots P(x_n \mid x_1, \ldots, x_{n-1})$$

Intuition: Break down the joint probability into conditional probabilities.

The **chain rule** decomposes a joint probability:

$$P(w_1, w_2, \ldots, w_n) = P(w_1) \cdot P(w_2 \mid w_1) \cdot P(w_3 \mid w_1, w_2) \ldots P(w_n \mid w_1, \ldots, w_{n-1})$$

Or more compactly:

$$P(W) = \prod_{i=1}^{n} P(w_1 \mid w_1, \ldots, w_{i-1})$$

Example: For "its water is so transparent that":

$$\begin{aligned} P(\text{its, water, is, so, transparent, that}) &= P(\text{its}) \\ &\times P(\text{water} \mid \text{its}) \\ &\times P(\text{its} \mid \text{its, water}) \\ &\times P(\text{so} \mid \text{its, water, is}) \\ &\times P(\text{transparent} \mid \text{its, water, is, so}) \\ &\times P(\text{that} \mid \text{its, water, is, so, transparent}) \end{aligned}$$

Why this helps: Conditional probabilities like $P(\text{water} \mid \text{its})$ are easier to estimate from data.

2.1.3 How to Estimate These Probabilities

Could we just count and divide?

No! The problem remains:

- **Too many possible sentences**: The number of possible word sequences is astronomical
- **Insufficient data**: We'll never see enough data to estimate probabilities for all possible sequences
- Even with the Chain Rule, long contexts like $P(\text{transparent} \mid \text{its, water, is, so})$ are still extremely rare

2.2 Neural Language Models

While N-gram models are simple and interpretable, they suffer from funamental limitations. Neural language models address these issues using deep learning to capture complex patterns in language.

2.2.1 Limitations of N-gram Models

N-gram models have several critical limitations:

- **Curse of dimensionality**: The number of parameters grows exponentially with n
- **Data sparsity**: Most N-grams never appear in training data
- **Lack of generalization**: Cannot handle unseen combinations of words
- **Fixed context**: Limited by the Markov assumption
- **No semantic understanding**: Treats words as discrete symbols

2.2.2 From N-grams to Neural Networks

Neural language models use continuous vector representation (embeddings) instead of discrete symbols:

Definizione 2.5 (Key innovation: Word Embeddings)

Dense vector representations capture semantic and syntactic relationship between words.
Example: Words like "cat", "dog", "kitten" have similar vector representations because they are semantically related.

2.2.3 Feedforward Neural Language Models

The first neural language models used feedforward netwroks to predict the next word.

Architecture:

1. **Input**: Concatenated embeddings of context words
2. **Hidden layers**: Learn complex transformations
3. **Output**: Probability distribution over vocabulary

Mathematica formulation:

$$P(w_i \mid w_{i-n+1}, \ldots, w_{i-1}) = \text{softmax}(W \cdot \tanh(U \cdot \mathbf{x} + b) + c)$$

Where $\mathbf{x}$ is the concatenated embedding vector of the context words.

2.2.4 Recurrent Neural Language Models

Definizione 2.6 (RNN Language Models)

RNNs process sequeces by maintaining a **hidden state** that captures information from previous timesteps:

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b)$$

$$P(w_t \mid w_{1:t-1}) = \text{softmax}(W_{hy} h_t + c)$$

Advantages:

- **Variable length**: Can handle sequences of any length
- **Memory**: Hidden state captures long-range dependencies
- **Shared parameters**: Same network process entire sequence

2.2.5 Long Short-Term Memory (LSTM)

Standard RNNs suffer from **vanishing gradients**, making it difficult to learn long-range dependencies. LSTMs solve this problem.

Teorema 2.7 (LSTM Architecture)

STMs use **gates** to control information flow:

- **Forget gate**: Decides what information to discard
- **Input gate**: Decides what new information to store
- **Output gate**: Decides what to output based on current state

LSTM equations:

$$\mathbf{f}_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$\mathbf{i}_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\widetilde{C_t} = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = \mathbf{f}_t \odot C_{t-1} + \mathbf{i}_t \odot \widetilde{C_t}$$

$$\mathbf{o}_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = \mathbf{o}_t \odot \tanh(C_t)$$